



Universidad Nacional Autónoma de México



Faculty of Engineering
Electrical Engineering Division

Course: Compilers **Group:** 5
Semester: 2026-2

Report: **Parser**

Professor:
M.C. René Adrián Dávila Pérez

Student IDs:

320060829
320317264
320206968
320025561
320266083

Delivery Date:
Sunday, May 28th, 2026

Contents

1	Introduction	2
2	Theorical Framework	3
3	Developmenmt	6
3.1	Overview of the implementation	6
3.2	Grammatic	6
3.3	Input/Output and execution	6
3.4	Relation with the propose CFG	6
3.5	Tests realized	6
4	Results	6
4.1	Input from keyboard	6
4.2	Input from file	6
4.3	Lexical Analysis in Terminal	6
4.4	Grouping of Tokens by Type	6
4.5	Code Visualization with Token Highlighting	6
4.6	Exporting Results	6
5	Conclusions	6
	References	6



1 Introduction

This document contains the necessary documentation for the comprehension of the construction of a syntactic analyzer and a Syntax-Directed Translation (SDT), which extend the lexical analyzer developed in the previous phase of this project. The system processes source code written in a subset of the C language, performing syntactic analysis through a bottom-up LALR(1) parser and semantic validation by means of SDT rules applied over the resulting Abstract Syntax Tree (AST).

The lexical phase, previously implemented using the Python Pygments library, produces a token flow that serves as input to the syntactic phase. Since PLY.YACC the tool selected for the LALR(1) parser, it requires a specific lexer interface, an adapter module was developed in order to integrate the two components. The parser then constructs an AST that is later traversed by the semantic analyzer, which enforces the typing and scoping rules of the language.

We should say that, as in the previous phase, the use of the FLEX and BISON tools is not permitted. Consequently, PLY (Python Lex-Yacc) was selected as the parser generator, which imposes specific requirements on both the grammar structure and the lexer output format, leading to a significant redesign of the Context-Free Grammar defined in the previous phase.

Objectives

- Extend the lexical analyzer developed in the previous phase into a full syntactic and semantic analysis for a subset of the C language.
- Adapt the Context-Free Grammar defined in the previous phase to be compatible with PLY.YACC's LALR(1) bottom-up parsing strategy, converting right-recursive productions to left-recursive form and delegating operator precedence to PLY's precedence mechanism.
- Construct an Abstract Syntax Tree (AST) during syntactic analysis by associating semantic actions with each grammar production rule, following the Syntax-Directed Translation approach.
- Implement a symbol table with scope management to enforce semantic rules, including variable declaration before use, type compatibility in assignments, valid operands in expressions, and correct argument usage in printf and scanf.
- Provide both a terminal-based and a Streamlit graphical interface for code input, analysis execution, and result visualization, including an interactive AST graph view.



2 Theoretical Framework

Firstly, the construction of a syntactic and semantic analyzer requires a solid understanding of several theoretical concepts that form the foundation of compiler design. [1]

In this project we address the intermediate stages of the front-end of the compiler: syntactic analysis and semantic validation through Syntax-Directed Translation. These phases work together to verify that the source code provided by the user is both structurally and semantically correct before proceeding with any code generation. As in the previous phase, the validation process is applied to code written in C that uses a specific set of instructions.

2.1 Syntactic Analysis (Parsing) and LALR(1)

The syntactic analyzer is the second stage of the compiler. It receives the token flow generated by the lexical analyzer and verifies that the sequence in order to accomplish the structural rules of the language, as defined by a Context-Free Grammar (CFG). [1]

In the previous phase of this project, a Top-Down Recursive Descent Parser was implemented, which constructs the parse tree starting from the initial grammar symbol and expanding downward toward the tokens. This approach required right-recursive production rules and explicit operator precedence layering within the grammar .

The current phase replaces this approach with a Bottom-Up LALR(1) parser, as implemented by PLY.YACC. In bottom-up parsing, the parse tree is constructed from the leaves (tokens) upward to the root (start symbol) through a sequence of shift and reduce operations. LALR(1) (Look-Ahead LR with 1 token of lookahead) is a widely used variant, it combines similar parser states to make a smaller and more efficient parsing table, while still working for most real programming languages.

A big advantage is that it handles left-recursive rules (which cause problems in top-down parsers) without issues in fact, they help make parsing faster.

Also, PLY.YACC lets you directly define operator precedence and associativity, so you don't have to complicate your grammar to handle them.

2.2 Grammar Adaptation for LALR(1)

A Context-Free Grammar (CFG) is a way to describe how a language is built. It is defined by four parts: $G=(V,T,P,S)$, where V is the set of non-terminal symbols (things that can be expanded), T is the set of terminal symbols (actual tokens of the language), P is the set of production rules written like $A \rightarrow \alpha$, and S is the start symbol. [3]

In the previous phase, the grammar used right-recursive rules to avoid left recursion, since top-down parsers cannot handle left recursion. For the LALR(1) parser, these rules were changed to left-recursive form, as recommended by PLY. For example:

$$\text{stmt_list} \rightarrow \text{stmt} \mid \text{stmt stmt_list}$$

(right-recursive, previous phase)


$$\text{stmt_list} \rightarrow \text{stmt_list stmt} \mid \text{stmt}$$

(left-recursive, current phase)

Also, operator precedence (like for arithmetic and logical operations) was removed from the grammar rules and instead defined using PLY's precedence tuple. This lets you specify priority and associativity (left, right, or none) in a simpler way, making the grammar cleaner. Finally, epsilon (empty) productions were used as little as possible. When they were necessary—such as in optional parts of a `for`-loop—an explicit empty rule was created using a non-terminal called `empty`, since PLY.YACC does not have a special way to write epsilon directly. [2]

2.3 Abstract Syntax Tree (AST)

The main goal of the parser is to take a sequence of tokens and organize them into a tree structure called an Abstract Syntax Tree (AST). Unlike a full parse tree, the AST removes unnecessary details like parentheses and punctuation, keeping only the important parts that give meaning to the program. This makes it easier to understand how the code is structured and to check that it follows the correct syntax. [1]

In this project, each node of the AST contains four things: the *kind* (the type of construct, like `"assign_stmt"` or `"if_stmt"`), the *children* (a list of child nodes or values), the *lineno* (the line number in the source code, useful for errors), and the *value* (the actual text for terminal symbols). The AST is built step by step during parsing, where each grammar rule creates its corresponding node using a function called `make_node`. [6]

2.4 Syntax-Directed Translation (SDT)

Syntax-Directed Translation (SDT) is a method where grammar rules are linked with actions that give them meaning. For each rule like $A \rightarrow \alpha$, some actions are executed when the parser uses that rule. These actions usually compute values (called synthesized attributes) that move from the child nodes up to the parent node. [1]

In this project, SDT is used in two steps. First, during parsing, each grammar rule creates an AST node as its result. Then, after parsing, a separate semantic analyzer goes through the AST and checks meanings by applying rules to each node. This two-step process separates building the structure from checking its correctness.

During semantic analysis, a detailed SDT trace is kept. For each node visited, it records the step number, the type of node, the line number, the action performed, and the state of the symbol table at that moment. This makes it easier to see what the program is doing and helps with debugging. [5]

2.5 Symbol Table and Scope Management

A symbol table is a data structure used during compilation to store information about variables (identifiers) in the program. At least, it keeps the name of each variable and its type.



It is used during semantic analysis to check that variables are declared before being used and that their types are used correctly. [1]

Scope management is handled using a stack of dictionaries, where each dictionary represents a different block of code. When the parser enters a new block (like inside an `if`, `while`, or `for`), a new scope is added to the stack. When leaving the block, that scope is removed. This makes sure that variables only exist inside the block where they were declared. When searching for a variable, the system looks from the most recent scope to the oldest one, following the usual scoping rules of C.[8]

2.6 Semantic Validation Rules

Semantic analysis checks that a program makes sense, even if it is written with correct syntax. In this project, the following simple rules are applied:

- **Declaration before use:** A variable must be declared before it is used. If not, an “undeclared variable” error is reported with the line number.
- **Type compatibility in assignment:** The value assigned to a variable must match its type. Types follow this order for compatibility: `char` $\leq$ `int` $\leq$ `float` $\leq$ `double`.
- **Valid operands in expressions:** Values used in expressions must be valid types like numbers or characters. If a variable is not declared, it causes an error that affects the whole expression.
- **printf and scanf argument validation:** The number and types of arguments must match the format specifiers. For example, `%d` is for `int`, `%f` for `float`/`double`, `%c` for `char`, and `%s` for strings. If they don’t match, an error is generated.
- **Evaluable conditions:** Conditions in `if`, `while`, and `for` must be of a type that can be evaluated as true or false, such as `bool`, `int`, `float`, `double`, or `char`.

2.7 Lexer Adapter and PLY Compatibility

PLY.YACC expects the lexer to have a `token()` method that returns one token at a time, where each token has `.type` and `.value`. When there are no more tokens, it should return `None`. However, the lexer used in this project (based on Pygments) gives a list of `(type, value)` pairs, which is not directly compatible.[7]

To fix this, an Adapter pattern was used. The class `PlyLexerAdapter` takes the list of tokens, converts the type names to the uppercase format expected by PLY, and provides a `token()` method that returns tokens one by one.

This way, the difference between the lexer and the parser is handled in one place, so both parts can work correctly without needing to change their original design. [4]



3 Developmenmt

3.1 Overview of the implementation

3.2 Grammatic

3.3 Input/Output and execution

3.4 Relation with the propose CFG

3.5 Tests realized

4 Results

4.1 Input from keyboard

4.2 Input from file

4.3 Lexical Analysis in Terminal

4.4 Grouping of Tokens by Type

4.5 Code Visualization with Token Highlighting

4.6 Exporting Results

5 Conclusions

References

- [1] A. Holub, *Compiler Design in C*. Prentice-Hall, 1990, ISBN: 0-13-155045-4. [Online]. Available: <https://holub.com/goodies/compiler/compilerDesignInC.pdf>
- [2] D. Beazley. "Ply (python lex-yacc) documentation." version 3.11. [Online]. Available: <https://www.dabeaz.com/ply/ply.html>
- [3] GeeksforGeeks. "What is context-free grammar? "[Online]. Available: <https://www.geeksforgeeks.org/theory-of-computation/what-is-context-free-grammar/>
- [4] GeeksforGeeks. "Lalr parser (look-ahead lr). "[Online]. Available: <https://www.geeksforgeeks.org/compiler-design/lalr-parser-with-examples/>
- [5] GeeksforGeeks. "Syntax directed translation in compiler design. "[Online]. Available: <https://www.geeksforgeeks.org/syntax-directed-translation-in-compiler-design/>



- [6] dev.to. “Abstract syntax tree (ast) deep dive: From theory to practical compiler implementation. ”[Online]. Available: https://dev.to/min_yi_e5fbf986e24f1c42df/abstract-syntax-tree-ast-deep-dive-from-theory-to-practical-compiler-implementation-4jpo
- [7] GeeksforGeeks. “Adapter design pattern. ”[Online]. Available: <https://www.geeksforgeeks.org/adapter-pattern/>
- [8] GeeksforGeeks. “Symbol table in compiler design. ”[Online]. Available: <https://www.geeksforgeeks.org/symbol-table-compiler/>
- [9] Pygments Contributors. “Builtin tokens,” Accessed: Mar. 13, 2026. [Online]. Available: <https://pygments.org/docs/tokens/>
- [10] GeeksforGeeks. “Ambiguous grammar. ”[Online]. Available: <https://www.geeksforgeeks.org/compiler-design/ambiguous-grammar/>
- [11] C. Wong. “Cfg developer. ”[Online]. Available: <https://web.stanford.edu/class/archive/cs/cs103/cs103.1156/tools/cfg/>